

ICM Fatturazioni

Emissione Fatture & Modifica Avvisi

La pagina-hub unica /avvisi-fattura

Documento tecnico-didattico dell'implementazione

Modulo	Ciclo di fatturazione — creazione avviso, modifica, generazione fattura
Pagina	Components/Pages/AvvisiFattura.razor (~1.400 righe)
Stack	.NET 10 · Blazor Server (InteractiveServer) · Dapper · SQL Server
Schema	ICMVerbali — migration 061-063 (fatt.Fatture + menu)
Test	455 unit test verdi · build 0 errori

1. Scopo e visione d'insieme

La pagina `/avvisi-fattura` è il **cuore operativo del ciclo di fatturazione**: è stata concepita come **pagina-hub unica** che concentra tre operazioni sulle stesse entità, evitando maschere separate quasi identiche. Da qui l'utente può:

- **Creare** un nuovo avviso di fattura per un'attività (selezionando le rate da fatturare);
- **Modificare** un avviso già emesso (testata, righe, spese) riusando esattamente gli stessi controlli della fase di creazione;
- **Generare la fattura** di cortesia da un avviso, con numerazione progressiva annuale e PDF.

La scelta di condensare tutto in una pagina nasce da un principio pratico: *modificare* un avviso e *crearlo* sono la stessa interazione sulle stesse tabelle. Tenerle separate avrebbe significato duplicare una UI complessa (griglia scadenze, spese, righe con riordino). Una pagina sola elimina la duplicazione al prezzo di un file più corposo — un trade-off consapevole.

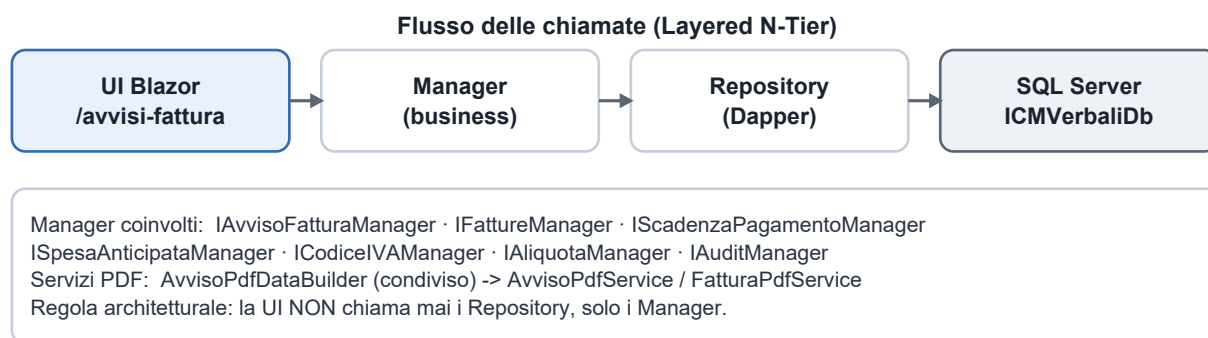


Fig. 1 — Architettura Layered N-Tier: la UI passa sempre dai Manager, mai dai Repository.

2. I due flussi della pagina

La pagina è una **macchina a stati** governata da un flag: `_idAvvisoInModifica`. Quando è null si sta **creando** un nuovo avviso; quando è valorizzato si sta **modificando** quell'avviso. Il campo `_avvisoOrig` conserva lo snapshot originale per copiare i valori immutabili al salvataggio.

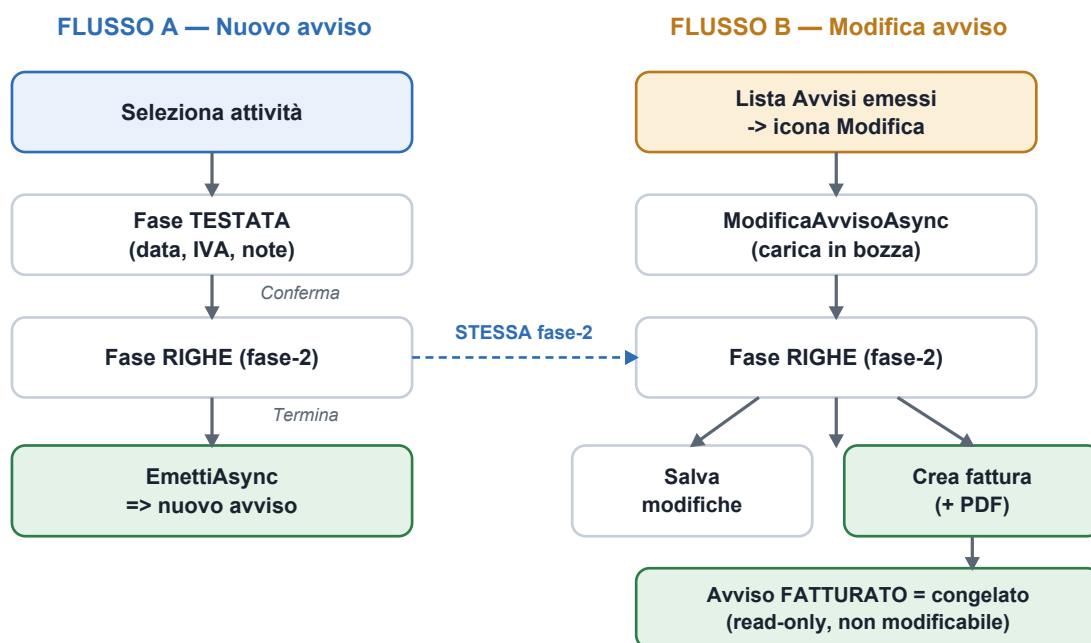


Fig. 2 — I due flussi convergono sulla STESSA fase-2 (righe): stesso markup, stessi handler.

Il punto chiave è la freccia tratteggiata al centro: la **fase-2** (griglia scadenze + spese + righe + anteprima cascata) è **un'unica istanza di markup**, condivisa dai due flussi. Il pulsante finale cambia solo etichetta e comportamento:

```
// Un solo pulsante, due comportamenti secondo lo stato
private async Task TerminaInserimentoAsync() {
    if (_idAvvisoInModifica is not null) { await SalvaModificheAsync(); return; }
    // ... altrimenti: validazioni + EmettiAsync (nuovo avviso) ...
}
```

3. Modello dati e vincoli

Il ciclo poggia su quattro tabelle dello schema fatt (dentro il DB unificato ICMVerbaLibDb). Un **avviso** è la testata; le sue **righe** sono le rate fatturate (o righe descrittive di solo testo); la **fattura** è il certificato 1:1 con numero e data.

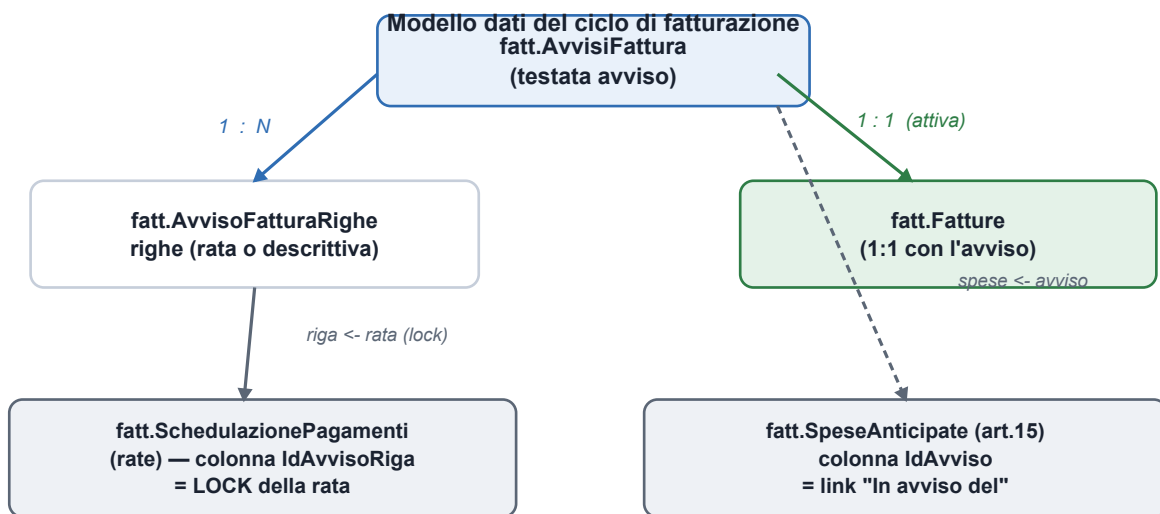


Fig. 3 — Entità e relazioni. Il lock della rata e il link della spesa sono colonne-marcatore.

Meccanismo	Colonna	Significato
Lock rata	SchedulazionePagamenti.IdAvvisoRiga	Una rata consumata da una riga-avviso è bloccata (non rifatturabile). NULL = libera.
Link spesa	SpeseAnticipate.IdAvviso	Spesa art.15 riaddebitata in un avviso. NULL = disponibile.
Unicità rata	UQ_AvvisoFatturaRighe_IdScadenza	Indice univoco filtrato: una scadenza in al più un avviso (anti doppia-fatturazione).
Unicità fattura	UQ_Fatture_IdAvviso_Activa	Un avviso ha al più UNA fattura attiva. L'annullo (soft-delete) la riapre.
Numerazione	UQ_Fatture_Anno_Numero_Activa	Numero univoco per anno solare fra le fatture attive.

Snapshot congelati. All'emissione l'avviso salva una copia delle aliquote (IVA, C.N.P.A.I.A., Ritenuta), del flag "applica ritenuta" (dal sostituto d'imposta del cliente) e dei riferimenti pagamento/banca. Così un avviso storico resta corretto anche se in seguito cambiano l'anagrafica o le aliquote vigenti.

4. La fase-2 riusata (righe dell'avviso)

È il blocco che l'utente vede sia creando sia modificando. Si compone di quattro sezioni:

- **Scadenze da fatturare** — griglia con checkbox, raggruppata per dettaglio, con le quattro grandezze live (Totale del dettaglio · In avvisi incl. corrente · Residuo). Si spuntano le rate e si premono "Aggiungi";
- **Spese anticipate (art.15)** — selezione delle spese da riaddebitare, con descrizione obbligatoria;
- **Righe dell'avviso** — la bozza ordinata (`_righeBozza`): rate reali e righe descrittive interlacciabili, con frecce su/giù per il riordino e ☐ per rimuovere;
- **Anteprima importi** — la cascata fiscale ricalcolata live (Imponibile → +C.N.P.A.I.A. → +IVA → -Ritenuta → +Spese → Totale nostro avere).

Un dettaglio importante è il calcolo del **"già allocato"** per riga: somma quanto è in avvisi precedenti PIÙ quanto è già nella bozza corrente, così il Residuo mostrato è sempre coerente con ciò che si sta componendo a video.

5. Modifica di un avviso esistente

Cliccando l'icona **Modifica** nella lista "Avvisi già emessi", ModificaAvvisoAsync carica l'avviso nella bozza (fase Righe) con quattro accortezze:

- **Righe pre-popolate:** le righe salvate diventano la bozza `_righeBozza`, nell'ordine memorizzato;
- **Spese:** si uniscono le spese libere e quelle già in questo avviso; queste ultime partono spuntate;
- **Scadenze disponibili** per aggiungere ALTRE rate: caricate **escludendo l'avviso corrente** dal conteggio del già-allocato (le sue rate sono già in bozza, non vanno contate due volte);
- **Numero fattura** proposto (ultima dell'anno + 1) e data odierna, pronti per la generazione.

L'esclusione dell'avviso corrente è realizzata con un parametro opzionale che si riflette nella query SQL:

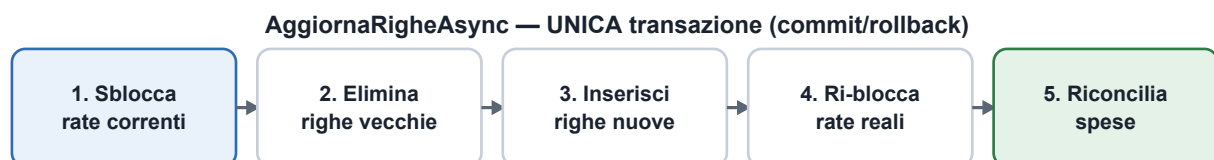
```
-- GetFatturabiliByAttivitaAsync(idAttivita, idAvvisoEscluso)
(SELECT COALESCE(SUM(r.Importo), 0)
 FROM fatt.AvvisoFatturaRighe r
 JOIN fatt.AvvisiFattura a ON a.IdAvviso = r.IdAvviso
 WHERE r.IdAttivitaDettaglio = d.IdAttivitaDettaglio
       AND a.IsAttivo = 1
       AND (@IdAvvisoEscluso IS NULL OR a.IdAvviso <> @IdAvvisoEscluso)
 ) AS GiaAllocatoAvvisiPrecedenti
```

Cosa è modificabile. In modifica si possono cambiare data, oggetto, note e **l'aliquota IVA**; si possono **aggiungere e togliere rate** e gestire spese e righe descrittive. C.N.P.A.I.A. e Ritenuta restano invece **snapshot congelati**: per cambiarle si annulla e si riemette l'avviso.

Al salvataggio, SalvaModificheAsync ricostruisce la testata copiando gli immutabili da `_avvisoOrig` e applica due operazioni: aggiornamento testata e sostituzione dei dettagli.

6. Aggiornamento dei dettagli: la strategia "replace-all"

Modificare le righe di un avviso significa toccare più tabelle in modo coerente: le righe stesse, i lock delle rate e i link delle spese. Per garantirne l'atomicità (pena la doppia fatturazione di una rata) l'operazione è un **aggregate write** in un'unica transazione: si **sostituisce l'intero insieme**.



Le rate/spese rimosse restano libere -> tornano fatturabili. Atomico: o tutto, o niente.

Fig. 4 — La sequenza transazionale di AggiornaRigheAsync. Un solo commit, altrimenti rollback.

Il Manager decide cosa scrivere e con quali importi autorevoli, il Repository esegue la sequenza. Le rate **mantenute** conservano importo/tipo dalla riga originale; quelle **aggiunte** li prendono dal read-model delle scadenze fatturabili (mai dai valori digitati a video):

```
// AggiornaDettagliAsync: fonte autorevole per ogni riga reale
if (correnti.TryGetValue(idScad, out var orig)) // rata mantenuta
    importo = orig.Importo; // dalla riga esistente
else if (fatturabili.TryGetValue(idScad, out var f)) // rata aggiunta
    importo = f.Importo; // dal read-model fatturabili
else throw new AvvisoFatturaInvalidaException(ScadenzaNonFatturabile);
```

Regola di contenuto. Un avviso deve avere almeno una rata OPPURE almeno una spesa (è ammesso l'avviso di sole spese art.15). Svuotarlo del tutto è vietato: per farlo si annulla l'avviso.

7. Generazione della fattura

Il pannello "Crea la fattura da questo avviso", in fondo alla modifica, propone il numero progressivo ($\text{ProponiNumeroAsync} = \text{MAX numero dell'anno} + 1$) e la data odierna. Il pulsante **Crea fattura** esegue tre passi in sequenza:

1	Salva le modifiche correnti (così la fattura riflette esattamente ciò che è a video).
2	Crea la fattura: valida (avviso attivo, non già fatturato, numero valido) e persiste con PK GUID v7.
3	Apri il PDF di cortesia in una nuova scheda e ricarica la lista (l'avviso diventa "Fatturato").

La correttezza sotto concorrenza è garantita a **doppia difesa**: il Manager fa un pre-check ("già fatturato?") per un messaggio chiaro, e l'**indice univoco filtrato** sul DB blocca comunque una seconda fattura sullo stesso avviso, traducendo l'errore SQL in eccezione di dominio.

```
public async Task<int> ProponiNumeroAsync(int anno, CancellationToken ct)
    => await _repo.GetMaxNumeroAnnoAsync(anno, ct) + 1;

// SviluppaFatturaAsync (UI): salva -> crea -> apre PDF -> esce
if (!await SalvaModificheAsync()) return; // blocca se validazione fallita
var id = await FattureManager.CreaAsync(new CreaFatturaRequest(idAvviso, num, data));
await JS.InvokeVoidAsync("open", $"/api/fatture/{id}/pdf", "_blank");
```

Stato "Fatturato" = congelato. Un avviso con una fattura attiva non è più modificabile né eliminabile (guardie nel Manager su Annulla/Testata/Dettagli): la fattura legge "live" dall'avviso, quindi modificarlo lo desincronizzerebbe dal PDF. Per riaprirlo si annullerà la fattura (dalla futura maschera di visualizzazione fatture).

Il PDF della fattura

Il documento riusa integralmente il layout dell'avviso: un **builder condiviso** (`AvvisoPdfDataBuilder`) assembla i dati, e `AvvisoPdfDocument` rende in "modalità fattura" cambiando solo titolo (FATTURA), la barra con numero/data fattura, il banner «DOCUMENTO NON VALIDO AI FINI FISCALI» e il riferimento all'avviso nel piè di pagina. Due servizi (`AvvisoPdfService` e `FatturaPdfService`) condividono lo stesso builder: nessuna duplicazione di layout.

8. Correttezza, sicurezza e tracciabilità

- **Doppia difesa** (Manager pre-check + sentinella/indice DB) su tutte le operazioni stateful: lock rate, unicità fattura, avviso già fatturato;
- **Transazionalità** replace-all per righe/lock/spese: o tutto o niente;
- **Importi autorevoli**: mai dai campi a video, sempre dal read-model o dalla riga originale;
- **Audit** (Regola 7): ogni creazione/modifica/eliminazione di avviso e fattura è tracciata in `fatt.Audit` con snapshot/diff JSON (best-effort, non blocca l'operazione);
- **Eccezioni tipizzate** di validazione (flusso previsto) NON loggate come errori; gli errori tecnici sì.

9. Mappa del codice

File	Ruolo
<code>Components/Pages/AvvisiFattura.razor</code>	UI e orchestrazione dei due flussi (~1.400 righe).
<code>Managers/AvvisoFatturaManager.cs</code>	Emissione, modifica testata/dettagli, annullo, calcolo cascata.

Managers/FattureManager.cs	Numerazione, creazione e annullo fattura, audit.
Repositories/AvvisoFatturaRepository.cs	Aggregate write: emissione, annullo, AggiornaRigheAsync (replace-all).
Repositories/FattureRepository.cs	CRUD fattura + guardie indici univoci.
Repositories/ScadenzaPagamentoRepository.cs	Read-model fatturabili (con esclusione avviso).
Services/AvvisoPdfDocument.cs	Layout PDF avviso/fattura (parametrico).
Services/AvvisoPdfDataBuilder.cs	Assemblaggio dati condiviso avviso/fattura.
ICMVerbali/Migrations/061-063	Schema fatt.Fatture + voce di menu (poi rimossa).

Nota di manutenzione: la pagina è coesa ma corposa. Se in futuro crescerà ancora, il candidato naturale al refactoring è estrarre la fase-2 (griglia scadenze + spese + righe + anteprima) in un componente condiviso `Components/Shared/AvvisoRigheEditor.razor`.